

5 Divide & Conquer

Efficient Algorithms · Wintersemester 2026–27

Prof. Dr. Sebastian Wild

CS566 (Wintersemester 2026–27)
Philipps-Universität Marburg
version 2026-11-10 14:08

Learning Outcomes

Unit 5: ***Divide & Conquer***

1. Know the steps of the Divide & Conquer paradigm.
2. Be able to solve simple Divide & Conquer recurrences.
3. Be able to design and analyze new algorithms using the Divide & Conquer paradigm.
4. Know the performance characteristics of selection-by-rank algorithms.
5. Know the divide and conquer approaches for integer multiplication, matrix multiplication, finding majority elements, and the closest-pair-of-points problem.

Outline

5 Divide & Conquer

5.1 Divide & Conquer Recurrences

5.2 Order Statistics

5.3 Linear-Time Selection

5.4 Fast Multiplication

5.5 Majority

5.6 Closest Pair of Points in the Plane

Divide and conquer

Divide and conquer *idiom* (Latin: *divide et impera*)
to make a group of people disagree and fight with one another
so that they will not join together against one (Merriam-Webster Dictionary)

⇝ in politics & algorithms, many independent, small problems are better than one big one!

Divide and conquer

Divide and conquer *idiom* (Latin: *divide et impera*)
to make a group of people disagree and fight with one another
so that they will not join together against one (Merriam-Webster Dictionary)

⇝ in politics & algorithms, many independent, small problems are better than one big one!

Divide-and-conquer algorithms:

1. Break problem into smaller, independent subproblems. (Divide!)
2. Recursively solve all subproblems. (Conquer!)
3. Assemble solution for original problem from solutions for subproblems.

Divide and conquer

Divide and conquer *idiom* (Latin: *divide et impera*)
to make a group of people disagree and fight with one another
so that they will not join together against one (Merriam-Webster Dictionary)

⇝ in politics & algorithms, many independent, small problems are better than one big one!

Divide-and-conquer algorithms:

1. Break problem into smaller, independent subproblems. (Divide!)
2. Recursively solve all subproblems. (Conquer!)
3. Assemble solution for original problem from solutions for subproblems.

Examples:

- Mergesort
- Quicksort
- Binary search
- (arguably) Tower of Hanoi

Clicker Question



Have you seen the *Master Method* before?

- **A** Sure, could apply it blindfolded
- **B** Vaguely remember
- **C** Never heard of it



→ *sli.do/effalg*

5.1 Divide & Conquer Recurrences

Back-of-the-envelope analysis

- before working out the details of a D&C idea,
  it is often useful to get a quick indication of the resulting performance
  - don't want to waste time on something that's not competitive in the end anyways!
- since D&C is naturally recursive, running time often not obvious
  instead: given by a recursive equation

Back-of-the-envelope analysis

- before working out the details of a D&C idea,
  it is often useful to get a quick indication of the resulting performance
  - don't want to waste time on something that's not competitive in the end anyways!
- since D&C is naturally recursive, running time often not obvious
  instead: given by a recursive equation
- unfortunately, rigorous analysis often tricky
  - Remember mergesort?

$$C(n) = \begin{cases} 0 & n \leq 1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + 2n & n \geq 2 \end{cases}$$

$$\rightsquigarrow \quad C(n) = 2n \lg(n) + \left(2 - \{\lg(n)\} - 2^{1-\{\lg(n)\}}\right)2n \quad 😱$$

$$= \Theta(n \log n) \quad 😌$$

Back-of-the-envelope analysis

- before working out the details of a D&C idea,
  it is often useful to get a quick indication of the resulting performance
  - don't want to waste time on something that's not competitive in the end anyways!
- since D&C is naturally recursive, running time often not obvious
  instead: given by a recursive equation
- unfortunately, rigorous analysis often tricky
  - Remember mergesort?

$$C(n) = \begin{cases} 0 & n \leq 1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + 2n & n \geq 2 \end{cases}$$

$$\rightsquigarrow \quad C(n) = 2n \lg(n) + \left(2 - \{\lg(n)\} - 2^{1-\{\lg(n)\}}\right)2n \quad 😱$$
$$= \Theta(n \log n) \quad 😌$$

- the following method works for many typical cases to give the right **order of growth**

The Master Method

- Assume a stereotypical D&C algorithm:
  - recursive calls on a subproblems of size n/b each (for $n > n_0$)
  - non-recursive "conquer" cost $f(n) > 0$ for $n > n_0$ (any $f : \mathbb{R}_{>0} \to \mathbb{R}_{>0}$)
  - base cases: $T(n) = d_n \geq 0$ for $n \leq n_0$ (arbitrary constants)

Mergesort

$a = 2$

$b = 2$

$f(n) = 2n$ // element visit

$n_0 = 1$

$d_0 = d_1 = 0$

The Master Method

- Assume a stereotypical D&C algorithm:
  - recursive calls on a subproblems of size n/b each (for $n > n_0$)
  - non-recursive "conquer" cost $f(n) > 0$ for $n > n_0$ (any $f : \mathbb{R}_{>0} \to \mathbb{R}_{>0}$)
  - base cases: $T(n) = d_n \geq 0$ for $n \leq n_0$ (arbitrary constants)

$\rightsquigarrow$ running time $T(n)$ satisfies

$$T(n) = \begin{cases} d_n & n \leq n_0 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n > n_0 \end{cases}$$

The Master Method

- Assume a stereotypical D&C algorithm:
  - recursive calls on a subproblems of size n/b each (for $n > n_0$)
  - non-recursive "conquer" cost $f(n) > 0$ for $n > n_0$ (any $f : \mathbb{R}_{>0} \to \mathbb{R}_{>0}$)
  - base cases: $T(n) = d_n \geq 0$ for $n \leq n_0$ (arbitrary constants)

$n = b^k$

$\rightsquigarrow$ running time $T(n)$ satisfies

$$T(n) = \begin{cases} d_n & n \leq n_0 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n > n_0 \end{cases}$$

$f(n)$ // $T(n) = 2T(n/4) + \sqrt{n}$.

$c = \log_b(a) = \log_4(2) = \frac{1}{2}$

Case (b)

$T(n) = \Theta(\sqrt{n} \log n)$

Theorem 5.1 (Master Theorem)

With $c := \log_b(a)$, we have for the above recurrence:

(a) $T(n) = \Theta(n^c)$ if $f(n) = O(n^{c-\varepsilon})$ for constant $\varepsilon > 0$.

(b) $T(n) = \Theta(n^c \log n)$ if $f(n) = \Theta(n^c)$.

(c) $T(n) = \Theta(f(n))$ if $f(n) = \Omega(n^{c+\varepsilon})$ for constant $\varepsilon > 0$ **and** f satisfies the *regularity condition* $\exists n_1, \rho < 1 \; \forall n \geq n_1 : \; a \cdot f\left(\frac{n}{b}\right) \leq \rho f(n)$.

Clicker Question

Solve the recurrence $T(n) = 2T(n/4) + \sqrt{n}$.



A $\Theta(\sqrt{n})$

B $\Theta(\sqrt{n} \log n)$

C $\Theta(n)$

D $\Theta(n^2)$

E $\Theta(\log n)$

F No idea



→ *sli.do/effalg*

Clicker Question

Solve the recurrence $T(n) = 2T(n/4) + \sqrt{n}$.



- **A** ~~$\Theta(\sqrt{n})$~~
- **B** $\Theta(\sqrt{n} \log n)$ ✓
- **C** ~~$\Theta(n)$~~
- **D** ~~$\Theta(n^2)$~~
- **E** ~~$\Theta(\log n)$~~
- **F** ~~No idea~~



→ *sli.do/effalg*

Master Theorem – Unfolding the recurrence

Assume $n = b^k$ (so $k = \log_b n$) and unfold: $T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$

Master Theorem – Unfolding the recurrence

Assume $n = b^k$ (so $k = \log_b n$) and unfold:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

$$= a\left(a\,T\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right)\right) + f(n)$$

Master Theorem – Unfolding the recurrence

Assume $n = b^k$ (so $k = \log_b n$) and unfold:

$$
\begin{aligned}
T(n) &= a\,T\left(\tfrac{n}{b}\right) + f(n) \\
&= a\left(a\,T\left(\tfrac{n}{b^2}\right) + f\left(\tfrac{n}{b}\right)\right) + f(n) \\
&\;\;\vdots \\
&= a^k\,T(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)
\end{aligned}
$$

Master Theorem – Unfolding the recurrence

Assume $n = b^k$ (so $k = \log_b n$) and unfold:

$$
\begin{aligned}
T(n) &= a\,T\left(\tfrac{n}{b}\right) + f(n) \\
&= a\left(a\,T\left(\tfrac{n}{b^2}\right) + f\left(\tfrac{n}{b}\right)\right) + f(n) \\
&\;\;\vdots \\
&= a^k\,T(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\
&= \underbrace{d \cdot n^{\log_b a}}_{\text{leaves}} + \underbrace{\sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)}_{\text{internal nodes, level by level}}
\end{aligned}
$$

(since $a^{\log_b n} = n^{\log_b a}$)

Master Theorem – Unfolding the recurrence

Assume $n = b^k$ (so $k = \log_b n$) and unfold:

$$\begin{aligned} T(n) &= a\,T\left(\tfrac{n}{b}\right) + f(n) \\ &= a\left(a\,T\left(\tfrac{n}{b^2}\right) + f\left(\tfrac{n}{b}\right)\right) + f(n) \\ &\;\;\vdots \\ &= a^k\,T(1) \;+\; \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \end{aligned}$$

(since $a^{\log_b n} = n^{\log_b a}$)

$$= \underbrace{d \cdot n^{\log_b a}}_{\text{leaves}} + \underbrace{\sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)}_{\text{internal nodes, level by level}}$$

$c = \log_b a$, $f(n) = n^\alpha$ $\rightsquigarrow$ summands $a^j \left(\frac{n}{b^j}\right)^\alpha = n^\alpha \left(\frac{a}{b^\alpha}\right)^j = n^\alpha\, b^{(c-\alpha)j}$: *geometric series!*

Master Theorem – Unfolding the recurrence

Assume $n = b^k$ (so $k = \log_b n$) and unfold:

$$\begin{aligned} T(n) &= a\,T\left(\tfrac{n}{b}\right) + f(n) \\ &= a\left(a\,T\left(\tfrac{n}{b^2}\right) + f\left(\tfrac{n}{b}\right)\right) + f(n) \\ &\;\;\vdots \\ &= a^k\,T(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &\text{(since } a^{\log_b n} = n^{\log_b a}\text{)} \quad = \underbrace{d \cdot n^{\log_b a}}_{\text{leaves}} + \underbrace{\sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)}_{\text{internal nodes, level by level}} \end{aligned}$$

$$\sum_{j=0}^{\log_b(n)} n^{\alpha} \cdot b^{\overbrace{(c-\alpha)j}^{>0}} \approx n^{\alpha} \cdot b^{(c-\alpha)\log_b(n)} \quad (\alpha < c) \quad = n^{\alpha} \cdot n^{c-\alpha} = n^{c}$$

$c = \log_b a$, $f(n) = n^{\alpha}$ $\rightsquigarrow$ summands $a^j \left(\frac{n}{b^j}\right)^{\alpha} = n^{\alpha}\left(\frac{a}{b^{\alpha}}\right)^{j} = n^{\alpha}\, b^{(c-\alpha)j}$: *geometric series!*

- $\alpha < c$: ratio > 1 $\rightsquigarrow$ last term dominates $\rightsquigarrow$ $\Theta(a^k) = \Theta(n^c)$ *(case 1: leaves win)*
- $\alpha = c$: ratio $= 1$ $\rightsquigarrow$ k equal terms $\rightsquigarrow$ $\Theta(n^c \log n)$ *(case 2: every level the same)*
- $\alpha > c$: ratio < 1 $\rightsquigarrow$ first term dominates $\rightsquigarrow$ $\Theta(f(n))$ *(case 3: root wins)*

The ε in cases 1 & 3 is what guarantees a ratio bounded away from 1;
the regularity condition in case 3 replaces the clean power n^{α} for general f.

Master Theorem – Intuition & Proof Idea

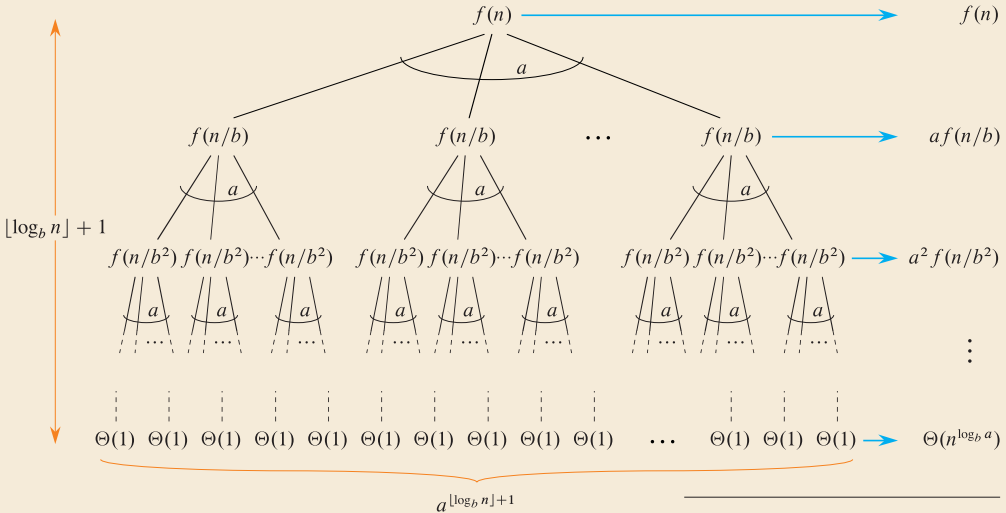
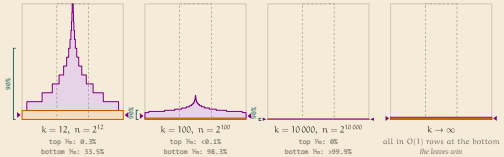


Figure 4.3 of Cormen et al. *Introduction to Algorithms* 4th ed.

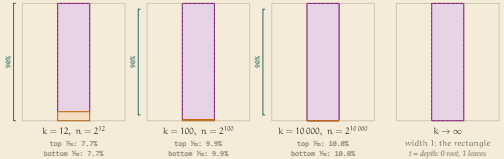
Master Theorem – Typical Examples

(a)



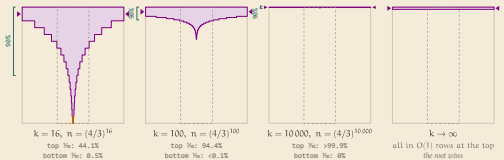
$$T(n) = \begin{cases} 3 \cdot T(n/2) + n & n > 1 \\ 1 & n \le 1 \end{cases}$$

(b)



$$T(n) = \begin{cases} 2 \cdot T(n/2) + n & n > 1 \\ 1 & n \le 1 \end{cases}$$

(c)



$$T(n) = \begin{cases} 1 \cdot T(3n/4) + 2n & n > 1 \\ 1 & n \le 1 \end{cases}$$

When it's fine to ignore floors and ceilings

Lemma 5.2 (Polynomial-growth master method)

If the toll function $f(n)$ satisfies the *polynomial-growth condition*, then the Θ-class of the solution of a D&C recurrence remains the same when ignoring floors and ceilings on subproblem sizes. ◀

$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 2n$

$T'(n) = T'(\frac{n}{2}) + T'(\frac{n}{2}) + 2n$

Campbell: *Floors and Ceilings in Divide-and-Conquer Recurrences*, ANALCO 2021

The ***polynomial-growth condition***

- $f : \mathbb{R}_{>0} \to \mathbb{R}$ satisfies the *polynomial-growth condition* if
  $\exists n_0\ \forall T \geq 1\ \exists D > 1 \quad \forall n \geq n_0\ \forall t \in [1, T] \ : \ \frac{1}{D} f(n) \leq f(tn) \leq D f(n)$

When it's fine to ignore floors and ceilings

Lemma 5.2 (Polynomial-growth master method)

If the toll function $f(n)$ satisfies the *polynomial-growth condition*,
then the Θ-class of the solution of a D&C recurrence remains the same
when ignoring floors and ceilings on subproblem sizes. ◀

Campbell: *Floors and Ceilings in Divide-and-Conquer Recurrences*, ANALCO 2021

The ***polynomial-growth condition***

- $f : \mathbb{R}_{>0} \to \mathbb{R}$ satisfies the *polynomial-growth condition* if
  $\exists n_0\ \forall T \geq 1\ \exists D > 1 \quad \forall n \geq n_0\ \forall t \in [1, T] \quad : \quad \frac{1}{D} f(n) \leq f(tn) \leq D f(n)$
- intuitively: increasing n by up to a factor T (and anywhere in between!)
  changes the function value by at most a factor $D = D(T)$ (for sufficiently large n)

When it's fine to ignore floors and ceilings

Lemma 5.2 (Polynomial-growth master method)

If the toll function $f(n)$ satisfies the *polynomial-growth condition*,
then the Θ-class of the solution of a D&C recurrence remains the same
when ignoring floors and ceilings on subproblem sizes. ◀

Campbell: *Floors and Ceilings in Divide-and-Conquer Recurrences*, ANALCO 2021

The ***polynomial-growth condition***

- $f : \mathbb{R}_{>0} \to \mathbb{R}$ satisfies the *polynomial-growth condition* if
  $\exists n_0\ \forall T \geq 1\ \exists D > 1 \quad \forall n \geq n_0\ \forall t \in [1, T] \quad : \quad \frac{1}{D} f(n) \leq f(tn) \leq D f(n)$
- intuitively: increasing n by up to a factor T (and anywhere in between!)
  changes the function value by at most a factor $D = D(T)$ (for sufficiently large n)

Note: This is orthogonal to the Case (c) regularity condition!

- but: $f(n) = \Theta\big(n^\alpha \log^\gamma(n) \log\log^\delta(n)\big)$ for constants α, γ, δ (0 allowed)
  $\rightsquigarrow$ f satisfies the polynomial-growth condition

if even $f(n) \sim B\, n^\alpha \log^\gamma(n) \log\log^\delta(n)$, $B > 0$, **and** $\alpha > c$
$\rightsquigarrow$ f satisfies polynomial-growth **and** the regularity condition

use this w/o proof

Clicker Question



Which case of the Master Theorem applies to $T(n) = 2T(n/2) + n \lg n$?

$f(n) = n \lg n$ satisfies poly. growth cond. & regularity condition

A Case 1, so $T(n) = \Theta(n)$

B Case 2, so $T(n) = \Theta(n \log n)$

C Case 3, so $T(n) = \Theta(n \log n)$

D None of the three cases applies



→ *sli.do/effalg*

Clicker Question



Which case of the Master Theorem applies to $T(n) = 2T(n/2) + n \lg n$?

- **A** ~~Case 1, so $T(n) = \Theta(n)$~~
- **B** ~~Case 2, so $T(n) = \Theta(n \log n)$~~
- **C** ~~Case 3, so $T(n) = \Theta(n \log n)$~~
- **D** None of the three cases applies ✓



→ *sli.do/effalg*

A Rigorous and Stronger Meta Theorem

∉ exam

Theorem 5.3 (Roura's Discrete Master Theorem)

Let $T(n)$ be recursively defined as

$$T(n) = \begin{cases} c_n & 0 \le n < n_0, \\ f(n) + \sum_{d=1}^{D} a_d \cdot T\left(\frac{n}{b_d} + r_{n,d}\right) & n \ge n_0, \end{cases}$$

where $D \in \mathbb{N}$, $a_d > 0$, $b_d > 1$, for $d = 1, \ldots, D$ are constants, functions $r_{n,d}$ satisfy $|r_{n,d}| = O(1)$ as $n \to \infty$, and function $f(n)$ satisfies $f(n) \sim B \cdot n^{\alpha}(\ln n)^{\gamma}$ for constants $B > 0$, α, γ.

A Rigorous and Stronger Meta Theorem

Theorem 5.3 (Roura's Discrete Master Theorem)

Let $T(n)$ be recursively defined as

$$T(n) = \begin{cases} c_n & 0 \leq n < n_0, \\ f(n) + \sum_{d=1}^{D} a_d \cdot T\left(\frac{n}{b_d} + r_{n,d}\right) & n \geq n_0, \end{cases}$$

where $D \in \mathbb{N}$, $a_d > 0$, $b_d > 1$, for $d = 1, \ldots, D$ are constants, functions $r_{n,d}$ satisfy $|r_{n,d}| = O(1)$ as $n \to \infty$, and function $f(n)$ satisfies $f(n) \sim B \cdot n^{\alpha}(\ln n)^{\gamma}$ for constants $B > 0$, α, γ.
Set $H = 1 - \sum_{d=1}^{D} a_d \, (1/b_d)^{\alpha}$; then we have:

A Rigorous and Stronger Meta Theorem

$T(n) = 2T(\frac{n}{2}) + n\log n$

$\alpha = 1 \quad \gamma = 1$

$H = 1 - 2\cdot\left(\frac{1}{2}\right)^1 = 0$

$T(n) \sim \frac{1}{\tilde{H}} \cdot n\log n \ln n$

Theorem 5.3 (Roura's Discrete Master Theorem)

Let $T(n)$ be recursively defined as

$$T(n) = \begin{cases} c_n & 0 \le n < n_0, \\ f(n) + \sum_{d=1}^{D} a_d \cdot T\left(\frac{n}{b_d} + r_{n,d}\right) & n \ge n_0, \end{cases}$$

where $D \in \mathbb{N}$, $a_d > 0$, $b_d > 1$, for $d = 1, \ldots, D$ are constants, functions $r_{n,d}$ satisfy $|r_{n,d}| = O(1)$ as $n \to \infty$, and function $f(n)$ satisfies $f(n) \sim B \cdot n^\alpha (\ln n)^\gamma$ for constants $B > 0$, α, γ.
Set $H = 1 - \sum_{d=1}^{D} a_d \, (1/b_d)^\alpha$; then we have:

- **(a)** If $H < 0$, then $T(n) = \Theta(n^{\tilde{\alpha}})$, for $\tilde{\alpha}$ the unique value of α that would make $H = 0$.
- **(b)** If $H = 0$ and $\gamma > -1$, then $T(n) \sim f(n)\ln(n)/\tilde{H}$ with constant $\tilde{H} = (\gamma+1)\sum_{d=1}^{D} a_d \, b_d^{-\alpha} \ln(b_d)$.
- **(c)** If $H = 0$ and $\gamma = -1$, then $T(n) \sim f(n)\ln(n)\ln(\ln(n))/\hat{H}$ with constant $\hat{H} = \sum_{d=1}^{D} a_d \, b_d^{-\alpha} \ln(b_d)$.
- **(d)** If $H = 0$ and $\gamma < -1$, then $T(n) = \Theta(n^\alpha)$.
- **(e)** If $H > 0$, then $T(n) \sim f(n)/H$.

5.2 Order Statistics

Selection by Rank

- Standard data summary of numerical data: (Data scientists, listen up!)
  - mean, standard deviation
  - min/max (range)
  - histograms

  } easy to compute in $\Theta(n)$ time

  - median, quartiles, other quantiles (a.k.a. order statistics)



computable in $\Theta(n)$ time?

Selection by Rank

- Standard data summary of numerical data: (Data scientists, listen up!)
  - mean, standard deviation
  - min/max (range)
  - histograms

  } easy to compute in $\Theta(n)$ time
  - median, quartiles, other quantiles (a.k.a. order statistics)



computable in $\Theta(n)$ time?

General form of problem: **Selection by Rank**

- **Given:** array $A[0..n)$ of numbers and number $k \in [0..n)$.
- **Goal:** find element that would be in position k if A was sorted (kth smallest element).

but 0-based & counting dups

- $k = \lfloor n/2 \rfloor \rightsquigarrow$ median; $k = \lfloor n/4 \rfloor \rightsquigarrow$ lower quartile
  $k = 0 \rightsquigarrow$ minimum; $k = n - \ell \rightsquigarrow$ ℓth largest

Easy: Sort A $\Theta(n \log n)$ then return $A[k]$

Quickselect

- Key observation: Finding the element of rank k seems hard.
  But computing the rank of a given element is easy!
  count smaller elements

Quickselect

- Key observation: Finding the element of rank k seems hard.
  But computing the rank of a given element is easy!
  count smaller elements

⇝ Pick any element $A[b]$ and find its rank j.

- $j = k$? ⇝ Lucky Duck! Return chosen element and stop
- $j < k$? ⇝ . . . not done yet. But: The $j + 1$ elements $\leq A[b]$ can be excluded!
- $j > k$? ⇝ similarly exclude the $n - j$ elements $\geq A[b]$

Quickselect

- Key observation: Finding the element of rank k seems hard.
  But computing the rank of a given element is easy!
  count smaller elements

⇝ Pick any element $A[b]$ and find its rank j.

- $j = k$? ⇝ Lucky Duck! Return chosen element and stop
- $j < k$? ⇝ ... not done yet. But: The $j + 1$ elements $\leq A[b]$ can be excluded!
- $j > k$? ⇝ similarly exclude the $n - j$ elements $\geq A[b]$

- partition function from Quicksort:
  - returns the rank of pivot
  - separates elements into smaller/larger

⇝ can use same building blocks

```
1  procedure quickselect(A[l..r), k):
2      if r − l ≤ 1 then return A[l] // assume k == 0
3      b := choosePivot(A[l..r))
4      j := partition(A[l..r), b)
5      if j == k
6          return A[j]
7      else if j < k
8          quickselect(A[j + 1..r), k)
9      else // j > k
10         quickselect(A[l..j), k)
```

Quickselect – Iterative Code

Recursion can be replaced by loop (*tail-recursion elimination*)

```
1  procedure quickselect(A[l..r), k):
2      if r − l ≤ 1 then return A[l]
3      b := choosePivot(A[l..r))
4      j := partition(A[l..r), b)
5      if j == k
6          return A[j]
7      else if j < k
8          quickselect(A[j + 1..r), k)
9      else // j > k
10         quickselect(A[l..j), k)
```

```
1  procedure quickselectIterative(A[0..n), k):
2      l := 0;  r := n
3      while r − l > 1
4          b := choosePivot(A[l..r))
5          j := partition(A[l..r), b)
6          if j ≥ k then r := j
7          if j ≤ k then l := j + 1
8      return A[k]
```

- implementations should usually prefer iterative version
- analysis more intuitive with recursive version

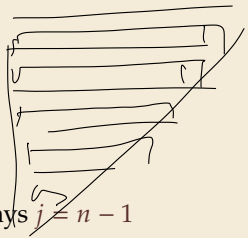
Quickselect – Analysis

```
1  procedure quickselect(A[l..r), k):
2      if r − l ≤ 1 then return A[l]
3      b := choosePivot(A[l..r))
4      j := partition(A[l..r), b)
5      if j == k
6          return A[j]
7      else if j < k
8          quickselect(A[j + 1..r), k)
9      else // j > k
10         quickselect(A[l..j), k)
```

- cost = #cmps = #element wird
- costs depend on n **and** k

Quickselect – Analysis

```
1  procedure quickselect(A[l..r), k):
2      if r − l ≤ 1 then return A[l]
3      b := choosePivot(A[l..r))
4      j := partition(A[l..r), b)
5      if j == k
6          return A[j]
7      else if j < k
8          quickselect(A[j + 1..r), k)
9      else // j > k
10         quickselect(A[l..j), k)
```



- cost = #cmps
- costs depend on n **and** k
- **worst case:** $k = 0$, but always $j = n - 1$
  - ⇝ each recursive call makes n one smaller at cost $\Theta(n)$
  - ⇝ $T(n, k) = \Theta(n^2)$ worst case cost

Quickselect – Analysis

```
1  procedure quickselect(A[l..r), k):
2      if r − l ≤ 1 then return A[l]
3      b := choosePivot(A[l..r))
4      j := partition(A[l..r), b)
5      if j == k
6          return A[j]
7      else if j < k
8          quickselect(A[j + 1..r), k)
9      else // j > k
10         quickselect(A[l..j), k)
```

- cost = #cmps
- costs depend on n **and** k
- **worst case:** $k = 0$, but always $j = n - 1$
  - $\rightsquigarrow$ each recursive call makes n one smaller at cost $\Theta(n)$
  - $\rightsquigarrow$ $T(n, k) = \Theta(n^2)$ worst case cost

average case:

- let $T(n, k)$ expected cost when we choose a pivot uniformly from $A[0..n)$

Quickselect – Analysis

```
1  procedure quickselect(A[l..r), k):
2      if r − l ≤ 1 then return A[l]
3      b := choosePivot(A[l..r))
4      j := partition(A[l..r), b)
5      if j == k
6          return A[j]
7      else if j < k
8          quickselect(A[j + 1..r), k)
9      else // j > k
10         quickselect(A[l..j), k)
```

- cost = #cmps
- costs depend on n **and** k
- **worst case:** $k = 0$, but always $j = n - 1$
  - ⇝ each recursive call makes n one smaller at cost $\Theta(n)$
  - ⇝ $T(n,k) = \Theta(n^2)$ worst case cost

$$[r=k] = \begin{cases} 1 & \text{if } r=k \\ 0 & \text{otherwise} \end{cases}$$

average case:

- let $T(n,k)$ expected cost when we choose a pivot uniformly from $A[0..n)$

⇝ formulate recurrence for $T(n,k)$ similar to BST/Quicksort recurrence

$$T(n,k) = n + \frac{1}{n}\sum_{r=0}^{n-1} [r=k]\cdot 0 + [k<r]\cdot T(r,k) + [k>r]\cdot T(n-r-1,k-r-1)$$

Quickselect – Average Case Analysis

- $T(n,k) = n + \frac{1}{n}\sum_{r=0}^{n-1}[r=k]\cdot 0 + [k<r]\cdot T(r,k) + [k>r]\cdot T(n-r-1,k-r-1)$
- Set $\hat{T}(n) = \max_{k\in[0..n)} T(n,k) \geqslant T(n,k)$

$$\leqslant \sum_{r=0}^{n-1} \max\{T(r,k), T(n-r-1,k-r-1)\}$$

$$\leqslant \sum_{r=0}^{n-1} \max\{\hat{T}(r), \hat{T}(n-r-1)\}$$

Quickselect – Average Case Analysis

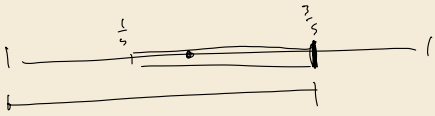
- $T(n,k) \;=\; n \;+\; \frac{1}{n}\sum_{r=0}^{n-1}[r=k]\cdot 0 \;+\; [k<r]\cdot T(r,k) \;+\; [k>r]\cdot T(n-r-1,k-r-1)$
- Set $\hat{T}(n) \;=\; \max_{k\in[0..n)} T(n,k)$

$$\rightsquigarrow\ \hat{T}(n) \;\leq\; n \;+\; \frac{1}{n}\sum_{r=0}^{n-1}\max\left\{\hat{T}(r),\hat{T}(n-r-1)\right\}$$

Quickselect – Average Case Analysis

- $T(n,k) \;=\; n \;+\; \frac{1}{n}\sum_{r=0}^{n-1}[r=k]\cdot 0 \;+\; [k<r]\cdot T(r,k) \;+\; [k>r]\cdot T(n-r-1,k-r-1)$
- Set $\hat{T}(n) \;=\; \max_{k\in[0..n)} T(n,k)$

$\rightsquigarrow\; \hat{T}(n) \;\leq\; n \;+\; \frac{1}{n}\sum_{r=0}^{n-1}\max\{\hat{T}(r),\hat{T}(n-r-1)\}$



- analyze hypothetical, worse algorithm:
  if $r \notin [\frac{1}{4}n, \frac{3}{4}n)$, discard partition and repeat with new pivot!

Quickselect Hmeans a Fandom

$\rightsquigarrow\; \hat{T}(n) \leq \tilde{T}(n)$ defined by $\quad \tilde{T}(n) \;\leq\; n \;+\; \frac{1}{2}\tilde{T}(n) \;+\; \frac{1}{2}\tilde{T}(\frac{3}{4}n)$ $\quad 1-\frac{1}{2}\tilde{T}(n)$ $\quad 1\cdot 2$

$\tilde{T}(n) \;\leq\; 2n \;+\; \tilde{T}(\frac{3}{4}n)$

Quickselect – Average Case Analysis

- $T(n,k) \;=\; n \;+\; \frac{1}{n}\sum_{r=0}^{n-1}[r=k]\cdot 0 \;+\; [k<r]\cdot T(r,k) \;+\; [k>r]\cdot T(n-r-1,k-r-1)$
- Set $\hat{T}(n) \;=\; \max_{k\in[0..n)} T(n,k)$

$\rightsquigarrow\; \hat{T}(n) \;\leq\; n \;+\; \frac{1}{n}\sum_{r=0}^{n-1}\max\{\hat{T}(r),\hat{T}(n-r-1)\}$

- analyze hypothetical, worse algorithm:
  if $r \notin [\frac{1}{4}n, \frac{3}{4}n)$, discard partition and repeat with new pivot!

$\rightsquigarrow\; \hat{T}(n) \;\leq\; \tilde{T}(n)$ defined by $\quad \tilde{T}(n) \;\leq\; n \;+\; \frac{1}{2}\tilde{T}(n) \;+\; \frac{1}{2}\tilde{T}(\frac{3}{4}n)$

$\rightsquigarrow\; \tilde{T}(n) \;\leq\; \underbrace{2n}_{f(n)} \;+\; \tilde{T}(\frac{3}{4}n)$ $\quad b = 4/3$

$a = 1$

$c = \log_b(a) = 0$ $\qquad f(n) = \Theta(n^c)$

$\Rightarrow\; \tilde{T}(n) = O(n)$

Quickselect – Average Case Analysis

- $T(n,k) \;=\; n \;+\; \frac{1}{n}\sum_{r=0}^{n-1}[r=k]\cdot 0 \;+\; [k<r]\cdot T(r,k) \;+\; [k>r]\cdot T(n-r-1,k-r-1)$
- Set $\hat{T}(n) \;=\; \max_{k\in[0..n)} T(n,k)$

$\rightsquigarrow\; \hat{T}(n) \;\leq\; n \;+\; \frac{1}{n}\sum_{r=0}^{n-1}\max\{\hat{T}(r),\hat{T}(n-r-1)\}$

- analyze hypothetical, worse algorithm:
  if $r \notin [\frac{1}{4}n, \frac{3}{4}n)$, discard partition and repeat with new pivot!

$\rightsquigarrow\; \hat{T}(n) \;\leq\; \tilde{T}(n)$ defined by $\qquad \tilde{T}(n) \;\leq\; n \;+\; \frac{1}{2}\tilde{T}(n) \;+\; \frac{1}{2}\tilde{T}(\frac{3}{4}n)$

$\rightsquigarrow\; \tilde{T}(n) \;\leq\; 2n \;+\; \tilde{T}(\frac{3}{4}n)$

- Master Theorem Case 3: $\tilde{T}(n) = \Theta(n)$

Quickselect Discussion

- $\Theta(n^2)$ worst case (like Quicksort)
- expected cost $\Theta(n)$ (best possible)
- no extra space needed
- adaptations possible to find several order statistics at once

Quickselect Discussion

- 👎 $\Theta(n^2)$ worst case (like Quicksort)
- 👍 expected cost $\Theta(n)$ (best possible)
- 👍 no extra space needed
- 👍 adaptations possible to find several order statistics at once

- 👍 expected cost can be further improved by choosing pivot from a small sorted sample
  - ⇝ asymptotically optimal randomized cost: $n + \min\{k, n-k\}$ comparisons in expectation
    achieved asymptotically by the *Floyd-Rivest algorithm*

5.3 Linear-Time Selection

Interlude – A recurring conversation

Cast of Characters:



Hi! I'm a *computer science practitioner*.
I love algorithms for the sometimes miraculous **applications** they enable.
I care for things I can **implement** and **that actually work in practice**.



Hi! I'm a *theoretical computer science researcher*.
I find beauty in elegant and **definitive** answers to questions about complexity.
I care for **eternal truths** and mathematically proven facts;
asymptotically optimal is what counts! (Constant factors are secondary.)

Quickselect Disagreements



`For practical purposes, (randomized) Quickselect is perfect.`

e. g. used in C++ STL `std::nth_element`

Quickselect Disagreements



`For practical purposes, (randomized) Quickselect is perfect.`

e. g. used in C++ STL `std::nth_element`

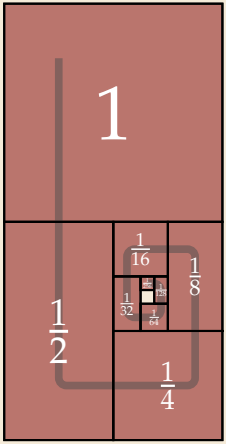


Yeah . . . maybe. But can we select by rank in $O(n)$ deterministic **worst case** time?

Better Pivots

It turns out, we can!

- All we need is better pivots!
  - If pivot was the exact median,
    we would at least halve #elements in each step
  - Then the total cost of all partitioning steps is $\leq 2n = \Theta(n)$.



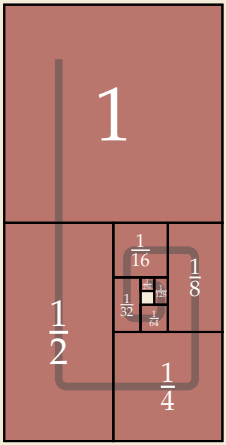
Better Pivots

It turns out, we can!

- All we need is better pivots!
  - If pivot was the exact median,
    we would at least halve #elements in each step
  - Then the total cost of all partitioning steps is $\leq 2n = \Theta(n)$.



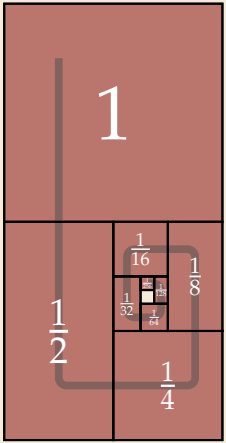
But: finding medians is (basically) our original problem!



Better Pivots

It turns out, we can!

- All we need is better pivots!
  - If pivot was the exact median,
    we would at least halve #elements in each step
  - Then the total cost of all partitioning steps is $\leq 2n = \Theta(n)$.

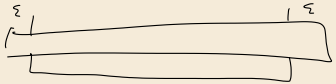




But: finding medians is (basically) our original problem!



It totally suffices to find an element of rank αn for $\alpha \in (\varepsilon, 1-\varepsilon)$
to get overall costs $\Theta(n)$!



The Median-of-Medians Algorithm

```
1  procedure choosePivotMoM(A[l..r)):
2      m := ⌊(r − l)/5⌋
3      for i := 0, . . . , m − 1
4          sort(A[l + 5i..l + 5i + 4])
5          // collect median of 5
6          Swap A[l + i] and A[l + 5i + 2]
7      return selectMoM(A[l..l + m), l + ⌊(m−1)/2⌋)
8
9  procedure selectMoM(A[l..r), k):
10     if r − l ≤ 5 then sort(A[l..r)); return k
11     b := choosePivotMoM(A[l..r))
12     j := partition(A[l..r), b)
13     if j == k
14         return j
15     else if j < k
16         return selectMoM(A[j + 1..r), k)
17     else // j > k
18         return selectMoM(A[l..j), k)
```

The Median-of-Medians Algorithm

```
1  procedure choosePivotMoM(A[l..r]):
2      m := ⌊(r − l)/5⌋
3      for i := 0, . . . , m − 1
4          sort(A[l + 5i..l + 5i + 4])
5          // collect median of 5
6          Swap A[l + i] and A[l + 5i + 2]
7      return selectMoM(A[l..l + m), l + ⌊(m−1)/2⌋)
8
9  procedure selectMoM(A[l..r), k):
10     if r − l ≤ 5 then sort(A[l..r)); return k
11     b := choosePivotMoM(A[l..r))
12     j := partition(A[l..r), b)
13     if j == k
14         return j
15     else if j < k
16         return selectMoM(A[j + 1..r), k)
17     else // j > k
18         return selectMoM(A[l..j), k)
```

Analysis:

- Note: 2 mutually recursive procedures
  - $\rightsquigarrow$ effectively 2 recursive calls!

1. recursive call inside choosePivotMoM on $m \leq \frac{n}{5}$ elements

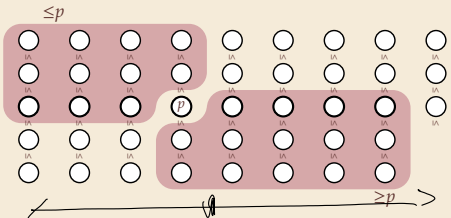
The Median-of-Medians Algorithm

```
1  procedure choosePivotMoM(A[l..r]):
2      m := ⌊(r − l)/5⌋
3      for i := 0, ..., m − 1
4          sort(A[l + 5i..l + 5i + 4])
5          // collect median of 5
6          Swap A[l + i] and A[l + 5i + 2]
7      return selectMoM(A[l..l + m), l + ⌊(m−1)/2⌋)
8
9  procedure selectMoM(A[l..r), k):
10     if r − l ≤ 5 then sort(A[l..r)); return k
11     b := choosePivotMoM(A[l..r))
12     j := partition(A[l..r), b)
13     if j == k
14         return j
15     else if j < k
16         return selectMoM(A[j + 1..r), k)
17     else // j > k
18         return selectMoM(A[l..j), k)
```

Analysis:

- Note: 2 mutually recursive procedures
  - ⇝ effectively 2 recursive calls!

1. recursive call inside choosePivotMoM on $m \leq \frac{n}{5}$ elements
2. recursive call inside selectMoM



⇝ partition excludes $\sim 3 \cdot \frac{m}{2} \sim \frac{3}{10}n$ elem.

$\geq \frac{3}{5} \cdot \frac{1}{2} \cdot n = \frac{3}{10} n$

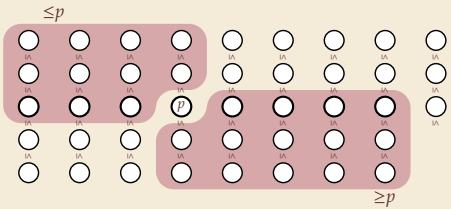
The Median-of-Medians Algorithm

```
1  procedure choosePivotMoM(A[l..r]):
2      m := ⌊(r − l)/5⌋
3      for i := 0, ..., m − 1
4          sort(A[l + 5i..l + 5i + 4])
5          // collect median of 5
6          Swap A[l + i] and A[l + 5i + 2]
7      return selectMoM(A[l..l + m), l + ⌊(m−1)/2⌋)
8
9  procedure selectMoM(A[l..r), k):
10     if r − l ≤ 5 then sort(A[l..r)); return k
11     b := choosePivotMoM(A[l..r))
12     j := partition(A[l..r), b)
13     if j == k
14         return j
15     else if j < k
16         return selectMoM(A[j + 1..r), k)
17     else // j > k
18         return selectMoM(A[l..j), k)
```

Analysis:

- Note: 2 mutually recursive procedures
  - $\rightsquigarrow$ effectively 2 recursive calls!

1. recursive call inside choosePivotMoM on $m \leq \frac{n}{5}$ elements
2. recursive call inside selectMoM



$\rightsquigarrow$ partition excludes $\sim 3 \cdot \frac{m}{2} \sim \frac{3}{10}n$ elem.

$\rightsquigarrow$ $C(n) \leq \Theta(n) + C(\frac{1}{5}n) + C(\frac{7}{10}n)$

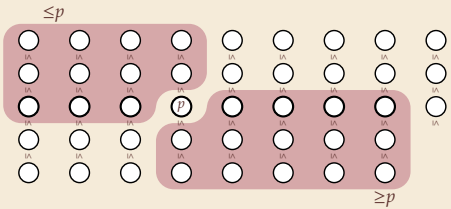
The Median-of-Medians Algorithm

```
1  procedure choosePivotMoM(A[l..r]):
2      m := ⌊(r − l)/5⌋
3      for i := 0, ..., m − 1
4          sort(A[l + 5i..l + 5i + 4])
5          // collect median of 5
6          Swap A[l + i] and A[l + 5i + 2]
7      return selectMoM(A[l..l + m), l + ⌊(m−1)/2⌋)
8
9  procedure selectMoM(A[l..r), k):
10     if r − l ≤ 5 then sort(A[l..r)); return k
11     b := choosePivotMoM(A[l..r))
12     j := partition(A[l..r), b)
13     if j == k
14         return j
15     else if j < k
16         return selectMoM(A[j + 1..r), k)
17     else // j > k
18         return selectMoM(A[l..j), k)
```

Analysis:

- Note: 2 mutually recursive procedures
  - ⇝ effectively 2 recursive calls!

1. recursive call inside choosePivotMoM on $m \leq \frac{n}{5}$ elements
2. recursive call inside selectMoM



⇝ partition excludes $\sim 3 \cdot \frac{m}{2} \sim \frac{3}{10}n$ elem.

$$\rightsquigarrow\ C(n) \leq \Theta(n) + C(\tfrac{1}{5}n) + C(\tfrac{7}{10}n)$$
$$\leq \Theta(n) + C(\tfrac{1}{5}n + \tfrac{7}{10}n)$$
$$= \Theta(n) + C(\tfrac{9}{10}n) \quad \rightsquigarrow \quad C(n) = \Theta(n)$$

ansatz: overall cost linear

Median of Medians – Analysis, rigorously

- Cost of `selectMoM` on n elements:
  $C(n) \le C\left(\frac{n}{5}\right) + C\left(\frac{7n}{10}\right) + f(n), \qquad f(n) = c \cdot n$
  (sort groups, rec. pivot, partition; $+O(1)$ rounding sizes)
- Two recursive calls of *different* sizes $\rightsquigarrow$ the Master Theorem does not apply.

Median of Medians – Analysis, rigorously

Exam

- Cost of `selectMoM` on n elements:
  $C(n) \le C\left(\frac{n}{5}\right) + C\left(\frac{7n}{10}\right) + f(n), \qquad f(n) = c \cdot n$
  (sort groups, rec. pivot, partition; $+O(1)$ rounding sizes)
- Two recursive calls of *different* sizes $\rightsquigarrow$ the Master Theorem does not apply.
- **Roura's Discrete Master Theorem** (slide "A Rigorous and Stronger Meta Theorem"):
  $D = 2, \quad a_1 = a_2 = 1, \quad z_1 = \frac{1}{5}, \quad z_2 = \frac{7}{10},$
  $t_n = cn \sim Bn^{\alpha}(\ln n)^{\gamma}$ with $\alpha = 1, \gamma = 0$

$$H = 1 - \sum_{d=1}^{D} a_d z_d^{\alpha} = 1 - \tfrac{1}{5} - \tfrac{7}{10} = \tfrac{1}{10} > 0 \qquad \rightsquigarrow \qquad C(n) \sim \frac{t_n}{H} = 10\,c\,n$$

$\rightsquigarrow$ $C(n) = \Theta(n)$, with the constant for free: the "$9/10$ of the work remains" heuristic becomes $\sum_k (9/10)^k = 10$.

Median of Medians – Analysis, rigorously

- Cost of `selectMoM` on n elements:
  $C(n) \le C\left(\frac{n}{5}\right) + C\left(\frac{7n}{10}\right) + f(n), \qquad f(n) = c \cdot n$ (sort groups, rec. pivot, partition; $+O(1)$ rounding sizes)
- Two recursive calls of *different* sizes $\rightsquigarrow$ the Master Theorem does not apply.
- **Roura's Discrete Master Theorem** (slide "A Rigorous and Stronger Meta Theorem"):
  $D = 2, \quad a_1 = a_2 = 1, \quad z_1 = \frac{1}{5}, \quad z_2 = \frac{7}{10},$
  $t_n = cn \sim Bn^\alpha (\ln n)^\gamma$ with $\alpha = 1, \gamma = 0$

$$H = 1 - \sum_{d=1}^{D} a_d z_d^\alpha = 1 - \tfrac{1}{5} - \tfrac{7}{10} = \tfrac{1}{10} > 0 \qquad \rightsquigarrow \qquad C(n) \sim \frac{t_n}{H} = 10\,c\,n$$

$\rightsquigarrow$ $C(n) = \Theta(n)$, with the constant for free: the "$9/10$ of the work remains" heuristic becomes $\sum_k (9/10)^k = 10$.

- Fine print: the $\le$ and the $\lceil\cdot\rceil$'s are covered – C is monotone, so we may analyze the recurrence with "=", and the DMT allows size perturbations $s_{d,n} = O(n^{1-\varepsilon})$.

5.4 Fast Multiplication

Clicker Question



How many **bit operations** does it take to multiply two n-bit integers?

A $O(1)$

B $O(\log \log n)$

C $O(\log n)$

D $O(\log^2 n)$

E $O(\sqrt{n})$

F $O(n)$

G $O(n \log n)$

H $O(n \log n \log \log n)$

I $O(n^2)$

J $O(n^2 \log n)$

K $O(n^3)$

L $O(2^n)$



→ *sli.do/effalg*

Integer Multiplication

- What's the cost of computing $x \cdot y$ for two integers x and y?

$\rightsquigarrow$ depends on how big the numbers are!

- If x and y have $O(w)$ bits, multiplication takes $O(1)$ time on word-RAM
- otherwise, need a dedicated algorithm!

Integer Multiplication

- What's the cost of computing $x \cdot y$ for two integers x and y?

$\rightsquigarrow$ depends on how big the numbers are!

- If x and y have $O(w)$ bits, multiplication takes $O(1)$ time on word-RAM
- otherwise, need a dedicated algorithm!

Long multiplication (»Schulmethode«)

easier in binary!
("shift and add")

```
1001010101 * 101101
-------------------
    1001010101
     0000000000
      1001010101
       1001010101
        0000000000
         1001010101
-------------------
    110100011110001
```

Integer Multiplication

- What's the cost of computing $x \cdot y$ for two integers x and y?

$\rightsquigarrow$ depends on how big the numbers are!

- If x and y have $O(w)$ bits, multiplication takes $O(1)$ time on word-RAM
- otherwise, need a dedicated algorithm!

Long multiplication (»Schulmethode«)

easier in binary!
("shift and add")

```
1001010101 * 101101
-------------------
    1001010101
     0000000000
      1001010101
       1001010101
        0000000000
         1001010101
-------------------
    110100011110001
```

- Given $x = \sum_{i=0}^{n-1} x_i 2^i$ and $y = \sum_{i=0}^{n-1} y_i 2^i$, want $z = \sum_{i=0}^{2n-1} z_i 2^i$

```
1 for i := 0,...,n-1
2     c := 0
3     for j := 0,...,n-1
4         z_{i+j} := z_{i+j} + c + x_i · y_j
5         c := ⌊z_{i+j}/2⌋
6         z_{i+j} := z_{i+j} mod 2
7     end for
8     z_{i+n} := c
9 end for
```

Integer Multiplication

- What's the cost of computing $x \cdot y$ for two integers x and y?

$\rightsquigarrow$ depends on how big the numbers are!

- If x and y have $O(w)$ bits, multiplication takes $O(1)$ time on word-RAM
- otherwise, need a dedicated algorithm!

Long multiplication (»Schulmethode«)

easier in binary!
("shift and add")

```
1001010101 * 101101
-------------------
    1001010101
     0000000000
      1001010101
       1001010101
        0000000000
         1001010101
-------------------
    110100011110001
```

- Given $x = \sum_{i=0}^{n-1} x_i 2^i$ and $y = \sum_{i=0}^{n-1} y_i 2^i$, want $z = \sum_{i=0}^{2n-1} z_i 2^i$

```
1 for i := 0,...,n-1
2     c := 0
3     for j := 0,...,n-1
4         z_{i+j} := z_{i+j} + c + x_i · y_j
5         c := ⌊z_{i+j}/2⌋
6         z_{i+j} := z_{i+j} mod 2
7     end for
8     z_{i+n} := c
9 end for
```

- $\Theta(n^2)$ bit operations
- could work with base 2^w instead of 2
  - $\rightsquigarrow$ $\Theta((n/w)^2)$ time
- here: count bit operations for simplicity
  can be generalized